

Fundamental Architecture

Table of Contents

1. [Shape System \(Bottom Layer - Pure Shape Definition\)](#)
 2. [MVC Layer \(Middle Layer - Node Business Logic\)](#)
 3. [Manager Layer \(Top Layer - All Nodes Management\)](#)
 4. [Editor \(Canvas Layer - SVG Container\)](#)
 5. [Correct Flow](#)
 6. [Rendering](#)
 7. [Integration](#)
-

Shape System

Bottom Layer - Pure Shape Definition

```
config.json (basic/rect/config.json)
├-> Defines: type, category, defaultWidth, defaultHeight, defaultPorts,
defaultStyle
└-> Loaded by ShapeLoader into ShapeRegistry

ShapeDefinition
├-> Wraps config.json data
└-> Stores metadata about a shape type

RectShape / CircleShape / DiamondShape
├-> BaseShape subclasses
├-> Has render(width, height, style) method
├-> Returns raw SVG element (just the shape geometry)
└-> Example: <rect>, <circle>, <path>

ShapeRegistry
├-> Stores all ShapeDefinitions
└-> getDefinition(type) → returns ShapeDefinition
```

Role: Define WHAT shapes look like (geometry only)

MVC Layer

Middle Layer - Node Business Logic

NodeModel

- └> Data: { id, type, x, y, width, height, label, style }
- └> Just a data object, no logic

NodeView

- └> Takes: NodeModel + ShapeDefinition
- └> Creates complete visual representation:
 - └> Gets shape SVG from shape.render()
 - └> Adds label <text>
 - └> Adds ports <circle>
 - └> Adds resize handles <rect>
 - └> Wraps in <g> group
- └> Returns complete node SVG group

NodeController

- └> Takes: NodeModel + NodeView
- └> Handles interactions:
 - └> Drag events
 - └> Resize events
 - └> Selection
 - └> Port clicks
- └> Updates NodeModel and NodeView

Role: Manage individual node instances (data + view + interaction)

Manager Layer

Top Layer - All Nodes Management

NodeManager

- └> Manages ALL nodes in the diagram
- └> Stores: Map<nodeId, {model, view, controller}>
- └> Methods:
 - └> createNode(type, x, y) → creates model+view+controller
 - └> getNode(id) → returns model
 - └> updateNode(id, data) → updates model & re-renders view
 - └> deleteNode(id) → removes everything
- └> Uses ShapeRegistry to get definitions

Role: CRUD operations for all nodes

Editor

Canvas Layer - SVG Container

Editor

- |> Manages the SVG canvas
- |> Has layers: grid, edges, nodes, overlay
- |> Methods:
 - |> addNodeElement(nodeId, svgGroup) → append to node layer
 - |> removeNodeElement(nodeId)
 - |> setViewport(x, y, zoom)
 - |> renderGrid()
- |> Does NOT create nodes itself

Role: Just the canvas container, delegates to NodeManager

Correct Flow

Creating a Node:

- [1] User drops shape from palette
 - ↓
- [2] app.js: nodeManager.createNode('rect', x, y)
 - ↓
- [3] NodeManager:
 - |> Create NodeModel({id, type:'rect', x, y, width, height})
 - |> Get ShapeDefinition from registry
 - |> Create NodeView(model, shapeDefinition)
 - |> NodeView gets RectShape.render() → <rect>
 - |> NodeView adds label, ports, handles
 - |> Returns complete <g> group
 - |> Create NodeController(model, view)
 - |> Attaches drag, resize, click handlers
 - |> editor.addNodeElement(nodeId, view.element)
 - ↓
- [4] Editor: Appends to node layer

Rendering

RectShape.render(width, height, style)
 ↳ Returns: <rect width="120" height="60" fill="#fff" stroke="#000"/>

NodeView (wraps it)

- |> <g class="node" transform="translate(x,y)">
 - |> <rect> ← from RectShape.render()
 - |> <text>Label</text>
 - |> <g class="ports">

```

    |
    |   |> <circle class="port" data-port-id="top"/>
    |   |> <circle class="port" data-port-id="right"/>
    |   |> ...
    |   |
    |   |> <g class="handles">
    |   |   |> <rect class="handle" data-handle-id="nw"/>
    |   |   |> ...
    |   |
    |   |> </g>
    |
    |> </g>

```

Integration

Complete Architecture Integration Guide

Overview

The corrected architecture properly separates concerns across multiple layers. Here's how everything fits together:



Component Roles & Responsibilities

1. Shape System (Data Layer)

config.json (e.g., **basic/rect/config.json**)

```

{
  "type": "rect",
  "category": "basic",
  "displayName": "Rectangle",
  "defaultWidth": 120,
  "defaultHeight": 60,
  "defaultPorts": [
    { "id": "top", "x": 60, "y": 0, "position": "top" },
    { "id": "right", "x": 120, "y": 30, "position": "right" },
    { "id": "bottom", "x": 60, "y": 60, "position": "bottom" },
    { "id": "left", "x": 0, "y": 30, "position": "left" }
  ],
  "defaultStyle": {
    "fill": "#ffffff",
    "stroke": "#424242",
    "strokeWidth": 2
  }
}

```

Role: Static configuration data for each shape type

ShapeDefinition class

```
class ShapeDefinition {
  constructor(config, shapeClass) {
    this.type = config.type;
    this.category = config.category;
    this.displayName = config.displayName;
    this.defaultWidth = config.defaultWidth;
    this.defaultHeight = config.defaultHeight;
    this.defaultPorts = config.defaultPorts;
    this.defaultStyle = config.defaultStyle;
    this.shapeClass = shapeClass; // ← RectShape class reference
  }
}
```

Role: Wraps config + shape class together

RectShape / CircleShape / DiamondShape classes

```
class RectShape extends BaseShape {
  static render(width, height, style) {
    const rect = document.createElementNS("http://www.w3.org/2000/svg",
"rect");
    rect.setAttribute("width", width);
    rect.setAttribute("height", height);
    rect.setAttribute("fill", style.fill || "#ffffff");
    rect.setAttribute("stroke", style.stroke || "#424242");
    rect.setAttribute("stroke-width", style.strokeWidth || 2);
    rect.setAttribute("rx", 4);
    return rect; // ← Just the <rect> element
  }
}
```

Role: Create the SVG geometry ONLY (no ports, no labels, no interactions)

ShapeRegistry

```
class ShapeRegistry {
  register(shapeDefinition) {
    this.shapes.set(shapeDefinition.type, shapeDefinition);
  }

  getDefinition(type) {
    return this.shapes.get(type); // Returns ShapeDefinition
  }
}
```

Role: Store all shape definitions, provide lookup

2. MVC Layer (Node Instance Layer)

NodeModel

```
class NodeModel {
  constructor(data) {
    this.id = data.id;
    this.type = data.type; // 'rect', 'circle', etc.
    this.x = data.x;
    this.y = data.y;
    this.width = data.width;
    this.height = data.height;
    this.label = data.label;
    this.style = data.style;
  }

  setPosition(x, y) {
    this.x = x;
    this.y = y;
  }
  setSize(w, h) {
    this.width = w;
    this.height = h;
  }
  setLabel(label) {
    this.label = label;
  }
}
```

Role: Pure data object for ONE node instance

NodeView

```
class NodeView {
  constructor(model, shapeDefinition) {
    this.model = model;
    this.shapeDefinition = shapeDefinition;
    this.element = this.createNodeGroup();
  }

  createNodeGroup() {
    const group = <g data-node-id="{model.id}" transform="translate(x,y)">

    // 1. Get shape from shape class
    const ShapeClass = this.shapeDefinition.shapeClass; // RectShape
    const shapeElement = ShapeClass.render(
```

```
        this.model.width,  
        this.model.height,  
        this.model.style  
    );  
    group.appendChild(shapeElement);  
  
    // 2. Add label  
    const label = <text>{this.model.label}</text>  
    group.appendChild(label);  
  
    // 3. Add ports (from shapeDefinition.defaultPorts)  
    const portsGroup = this.createPortsGroup();  
    group.appendChild(portsGroup);  
  
    // 4. Add resize handles  
    const handlesGroup = this.createHandlesGroup();  
    group.appendChild(handlesGroup);  
  
    return group;  
}  
  
update() {  
    // Update transform, size, label, ports, handles when model changes  
}  
}
```

Role:

- Create complete visual representation
- Use shape.render() for geometry
- Add ports, labels, handles
- Update view when model changes

NodeController

```
class NodeController {  
    constructor(model, view, eventBus) {  
        this.model = model;  
        this.view = view;  
        this.eventBus = eventBus;  
        this.setupEventHandlers();  
    }  
  
    setupEventHandlers() {  
        // Attach listeners to view.element  
        const shapeElement = this.view.element.querySelector(".node-shape");  
  
        shapeElement.addEventListener("mousedown", (e) =>  
this.handleDragStart(e));  
        // ... more handlers  
    }  
}
```

```
}

handleDragStart(e) {
  // Track drag
}

handleDragMove(e) {
  // Update model.x, model.y
  this.model.setPosition(newX, newY);

  // Update view transform
  this.view.element.setAttribute("transform", `translate(${newX},
${newY})`);

  // Emit event
  this.eventBus.emit("node:dragging", {
    nodeId: this.model.id,
    x: newX,
    y: newY,
  });
}

handlePortMouseDown(e) {
  const portId = e.target.getAttribute("data-port-id");
  this.eventBus.emit("port:mousedown", {
    nodeId: this.model.id,
    portId: portId,
  });
}
}
```

Role:

- Handle ALL interactions (drag, resize, port clicks)
- Update model based on interactions
- Emit events via EventBus

3. Manager Layer (All Nodes Management)

NodeManager

```
class NodeManager {
  constructor(editor, shapeRegistry, eventBus, stateManager) {
    this.editor = editor;
    this.shapeRegistry = shapeRegistry;
    this.eventBus = eventBus;
    this.nodes = new Map(); // Map<nodeId, {model, view, controller}>
  }

  createNode(type, x, y, options) {
```



```

// 1. Get shape definition
const shapeDefinition = this.shapeRegistry.getDefinition(type);

// 2. Create model
const model = new NodeModel({
  id: `node_${Date.now()}`,
  type: type,
  x: x,
  y: y,
  width: shapeDefinition.defaultWidth,
  height: shapeDefinition.defaultHeight,
  label: options.label || type,
  style: shapeDefinition.defaultStyle,
});

// 3. Create view (uses shapeDefinition to get shape class)
const view = new NodeView(model, shapeDefinition);

// 4. Create controller
const controller = new NodeController(model, view, this.eventBus);

// 5. Store MVC components
this.nodes.set(model.id, { model, view, controller });

// 6. Add to editor canvas
this.editor.addNodeElement(model.id, view.element);

// 7. Emit event
this.eventBus.emit("node:created", { nodeId: model.id });

return model.id;
}

updateNode(nodeId, updates) {
  const { model, view } = this.nodes.get(nodeId);
  if (updates.x !== undefined) model.setPosition(updates.x, updates.y);
  if (updates.width !== undefined)
    model.setSize(updates.width, updates.height);
  view.update(); // Re-render
}

removeNode(nodeId) {
  const { view, controller } = this.nodes.get(nodeId);
  this.editor.removeNodeElement(nodeId);
  controller.destroy();
  view.destroy();
  this.nodes.delete(nodeId);
}
}

```

Role:

- CRUD for all nodes

- Orchestrate Model + View + Controller creation
- Delegate canvas operations to Editor

4. Canvas Layer (SVG Container)

Editor

```
class Editor {
  constructor(eventBus, stateManager, container) {
    this.eventBus = eventBus;
    this.container = container; // DOM element
    this.nodeLayer = null;
    this.edgeLayer = null;
  }

  initialize() {
    this.createSVGStructure();
    this.setupEventHandlers();
  }

  createSVGStructure() {
    this.svg = (
      <svg>
        <defs>
          <marker id="arrowhead">...</marker>
        </defs>
        <g id="viewport-group">
          <g id="grid-layer"></g>
          <g id="edge-layer"></g>
          <g id="node-layer"></g> ← Nodes go here
          <g id="overlay-layer"></g>
        </g>
      </svg>
    );

    this.container.appendChild(this.svg);
  }

  // NodeManager calls these:
  addNodeElement(nodeId, nodeElement) {
    this.nodeLayer.appendChild(nodeElement);
  }

  removeNodeElement(nodeId) {
    const element = this.nodeLayer.querySelector(`[data-node-id="${nodeId}"]`);
    this.nodeLayer.removeChild(element);
  }

  // Canvas-level interactions (pan, zoom)
  handleWheel(e) {
```

```

    /* zoom */
  }
  handleMouseDown(e) {
    /* pan */
  }
  handleDrop(e) {
    // Emit 'shape:dropped' event
    this.eventBus.emit("shape:dropped", { type, x, y });
  }
}

```

Role:

- Just the SVG container
- Provide add/remove element methods
- Handle canvas interactions (zoom/pan)
- Does NOT create nodes itself

Complete Integration Flow

Initialization Flow:

```

[1] app.js → new FlowchartApp()
    ↓
[2] ServiceProvider.register(container)
    ├──> Register EventBus
    ├──> Register ShapeRegistry
    ├──> Register Editor
    ├──> Register NodeManager (with Editor + ShapeRegistry)
    └──> Register StateManager
    ↓
[3] ShapeLoader.loadBuiltInShapes(shapeRegistry)
    ├──> Load basic/rect/config.json
    ├──> Import RectShape class
    ├──> Create ShapeDefinition(config, RectShape)
    └──> shapeRegistry.register(shapeDefinition)
    ↓
[4] editor.initialize()
    └──> Create SVG layers
    ↓
[5] Setup event handlers in app.js

```

Creating a Node Flow:

```

[USER] Drops shape from palette
    ↓
[1] Editor.handleDrop()

```

```

└─> EventBus.emit('shape:dropped', { type: 'rect', x: 100, y: 100 })
↓
[2] app.js event handler receives 'shape:dropped'
└─> nodeManager.createNode('rect', 100, 100)
↓
[3] NodeManager.createNode()
├─> shapeDefinition = shapeRegistry.getDefinition('rect')
│   └─> Returns ShapeDefinition with RectShape class
├─> model = new NodeModel({ type: 'rect', x: 100, y: 100, ... })
├─> view = new NodeView(model, shapeDefinition)
│   └─> NodeView.createNodeGroup()
│       ├──> ShapeClass = shapeDefinition.shapeClass (RectShape)
│       ├──> shapeElement = RectShape.render(width, height, style)
│       │   └─> Returns <rect width="120" height="60" .../>
│       ├──> Add <text> label
│       ├──> Add ports from shapeDefinition.defaultPorts
│       ├──> Add resize handles
│       └─> Returns complete <g> with everything
├─> controller = new NodeController(model, view, EventBus)
│   └─> Attach event listeners to view.element
├─> nodes.set(nodeId, { model, view, controller })
└─> editor.addNodeElement(nodeId, view.element)
    └─> Appends view.element to nodeLayer

```

Final SVG:

```

<svg>
  <g id="node-layer">
    <g class="node" data-node-id="node_123" transform="translate(100,
100)">
      <rect width="120" height="60" fill="#fff" stroke="#424242"/> ← From
RectShape.render()
      <text>Rect</text> ← From
NodeView
      <g class="ports-group"> ←
From NodeView
        <circle class="port" data-port-id="top" cx="60" cy="0"/>
        <circle class="port" data-port-id="right" cx="120" cy="30"/>
        ...
      </g>
      <g class="handles-group"> ←
From NodeView
        <rect class="resize-handle" data-handle-id="nw"/>
        ...
      </g>
    </g>
  </g>

```

```
</g>  
</svg>
```

User Drags Node Flow:

```
[USER] Clicks on node shape and drags  
↓  
[1] NodeController.handleDragStart()  
    ↳ Attach document.mousemove listener  
↓  
[2] NodeController.handleDragMove()  
    ↳ Calculate newX, newY  
    ↳ model.setPosition(newX, newY)  
    ↳ view.element.setAttribute('transform', `translate(${newX},  
    ${newY})`)  
    ↳ EventBus.emit('node:dragging', { nodeId, x, y })  
↓  
[3] NodeController.handleDragEnd()  
    ↳ EventBus.emit('node:moved', { nodeId, x, y })  
↓  
[4] app.js or EdgeManager can listen to 'node:moved'  
    ↳ Update connected edges
```

User Clicks Port Flow:

```
[USER] Clicks on port circle  
↓  
[1] NodeController.handlePortMouseDown()  
    ↳ EventBus.emit('port:mousedown', { nodeId, portId, portPosition })  
↓  
[2] app.js event handler receives 'port:mousedown'  
    ↳ stateManager.setViewportMode('connecting')  
    ↳ Store: connectingFrom = { nodeId, portId }  
↓  
[3] User clicks on another port  
↓  
[4] app.js creates edge  
    ↳ edgeManager.createEdge(sourceNodeId, targetNodeId, sourcePortId,  
    targetPortId)
```

Key Differences from Old Code

✗ Old (Wrong) Way:

```
// Editor.js was doing EVERYTHING:
Editor.renderNode(nodeId, nodeData, shapeDefinition) {
  // ❌ Editor creates shape geometry itself
  const rect = document.createElementNS('svg', 'rect');
  rect.setAttribute('width', nodeData.width);

  // ❌ Editor creates ports itself
  const port = document.createElementNS('svg', 'circle');

  // ❌ Editor attaches event handlers itself
  rect.addEventListener('mousedown', (e) => { /* drag logic */ });

  // ❌ Bypasses NodeManager, NodeModel, NodeView, NodeController
  completely
}

// app.js was calling Editor directly:
app.js:
  this.editor.renderNode(nodeId, nodeData, shapeDefinition); // ❌ Wrong!
```

✅ New (Correct) Way:

```
// Shape creates its own geometry:
RectShape.render(width, height, style) {
  return <rect>; // ✓ Shape knows how to draw itself
}

// NodeView uses shape to build complete node:
NodeView.createNodeGroup() {
  const shapeElement = RectShape.render(...); // ✓ Delegate to shape
  // Add ports, labels, handles
  return <g>...</g>; // ✓ Complete node
}

// NodeController handles interactions:
NodeController.setupEventHandlers() {
  shapeElement.addEventListener(...); // ✓ Controller handles interactions
}

// NodeManager orchestrates everything:
NodeManager.createNode(type, x, y) {
  const model = new NodeModel(...);
  const view = new NodeView(model, shapeDefinition);
  const controller = new NodeController(model, view);
  editor.addNodeElement(view.element); // ✓ Just add to canvas
}

// app.js calls NodeManager:
app.js:
  this.nodeManager.createNode('rect', x, y); // ✓ Correct!
```

Summary

Separation of Concerns:

Component	Responsibility
<code>config.json</code>	Static shape data
<code>ShapeDefinition</code>	Wrap config + shape class
<code>RectShape/CircleShape</code>	Create SVG geometry ONLY
<code>ShapeRegistry</code>	Store all definitions
<code>NodeModel</code>	Data for ONE node
<code>NodeView</code>	Visual for ONE node (uses <code>shape.render()</code>)
<code>NodeController</code>	Interactions for ONE node
<code>NodeManager</code>	CRUD for ALL nodes
<code>Editor</code>	SVG canvas container

Data Flow:



This architecture is:

-  Maintainable (each class has one job)
-  Extensible (easy to add new shapes)
-  Testable (can test each piece independently)
-  Reusable (NodeView works with any shape)
-  Clean (no god classes doing everything)

Now all components work in perfect coordination! 🎉